der (RNN + Attention)

Project	Seq2Seq Review Summarizer with RNN + Attention
Step	Step 5 of 8 — Decoder (RNN + Attention)
Depends On	Step 1 (vocabulary.pkl) + Step 2 (DataLoaders) + Step 3 (Encoder) + Step 4 (Attention)
Status	COMPLETE — All 11 verification checks passed
Architecture	Embedding → Dropout → Unidirectional GRU (2 layers) → FC Projection
Decoder Params	13,870,307 (dominates due to fc_out: 256 → 33,379)
Output to Step 6	predictions (tgt_len-1, batch, vocab_size) + attn_weights (tgt_len-1, batch, 81)
Date	February 2026

1. Overview

This document records all work completed in Step 5 — the Decoder. The Decoder is the second and final neural network component in the Seq2Seq architecture. Its job is to receive the compressed summary of a review from the Encoder and generate a short, readable title word by word. At every generation step, it consults the Attention mechanism to decide which parts of the original review are most relevant right now.

Unlike the Encoder which reads the entire source sequence in one forward pass, the Decoder works sequentially — one token at a time. It starts with a special Start-Of-Sequence token and keeps predicting the next word until it either emits an End-Of-Sequence token or reaches the maximum title length of 12 tokens.

1.1 What the Decoder Does in Plain Terms

Imagine you have just read a long review and taken detailed notes. You now need to write a one-line title. You do not write the whole title at once — you write it one word at a time. Before writing each word, you glance back at your notes to find the most relevant part. That glancing back is the Attention mechanism. The word you actually write is what the Decoder produces.

The Decoder maintains a 'memory' of what it has written so far (its GRU hidden state). At each step it combines this memory with what the Attention says is currently important in the source, and uses that combined signal to predict the single most likely next word from the full vocabulary of 33,379 tokens.

2. Architecture — Single-Step Decoder Forward Pass

The Decoder is called once per title word position. Each call takes the current decoder hidden state and one input token, and returns the next word prediction plus an updated hidden state. The loop across all 11 positions (MAX_TGT_LEN – 1 = 12 – 1) is managed by the Seq2Seq wrapper in Step 6.

2.1 The Five Steps Inside One Decoder Forward Call

Step	Operation	Shape In → Out
1	Embed input token with dropout	(batch,) → (batch, 1, 128)
2	Attention: score all 81 encoder positions using top GRU layer hidden state	(batch, 256) → attn_w (batch, 81), ctx (batch, 256)
3	Concatenate embedding + context vector	(batch,1,128) + (batch,1,256) → (batch, 1, 384)
4	One GRU step — update decoder memory	(batch, 1, 384) → output (batch, 1, 256), hidden (2, batch, 256)
5	FC projection → raw logits over full vocabulary	(batch, 256) → (batch, 33,379)

2.2 Key Design Decisions

Decision	Choice	Why
Direction	Unidirectional GRU	Decoder generates left-to-right — it cannot see future words it hasn't written yet
GRU Input	Embedding + Context (384 dim)	Context vector from Attention is concatenated before GRU — not just at the start. This is the Bahdanau standard
Attention query	Top GRU layer hidden state only	The top layer has the richest abstract representation of what the decoder is 'thinking about'
Output	Raw logits (no softmax)	nn.CrossEntropyLoss applies log-softmax internally — numerically more stable than explicit softmax
Teacher Forcing	Ratio = 0.5	50% of steps use ground-truth previous token. Implemented in Step 6 Seq2Seq wrapper, not inside Decoder
Shared Vocab	Same Embedding as Encoder	Both encoder and decoder use the same 33,379-token vocabulary and embedding size of 128

2.3 Architecture Summary Table

Component	Shape / Detail
Input: tgt_token	(batch=64,) — one token index per sample
Input: decoder_hidden	(num_layers=2, batch=64, hidden_dim=256) — initialised from encoder final hidden
Input: encoder_outputs	(batch=64, src_len=81, hidden_dim=256) — all encoder states from Step 3
Input: src_lens	(batch=64,) — passed through to Attention for PAD masking
Embedding (Linear)	33,379 × 128 — same vocab/dim as encoder, PAD index → zero vector
Dropout	p=0.5 — applied to embedding output only
GRU	input=384, hidden=256, layers=2, unidirectional, batch_first=True
fc_out (Linear)	256 → 33,379 with bias — projects GRU output to full vocabulary logits

Output: prediction	(batch=64, vocab_size=33,379) — raw logits, pass to CrossEntropyLoss
Output: decoder_hidden	(2, batch=64, 256) — updated hidden state for the next decoder step
Output: attn_weights	(batch=64, src_len=81) — attention distribution for this step, saved for Step 8 viz
Total Decoder Params	13,870,307 (fc_out alone: 256×33,379 + 33,379 = ~8.6M)

3. Parameter Count Breakdown

The Decoder is the largest module in the architecture at 13.87M parameters, compared to 6.18M for the Encoder and 131K for the Attention module. This asymmetry is entirely due to the fc_out projection layer which must map from 256 dimensions to every word in the 33,379-token vocabulary.

Module	Parameters	Breakdown
Encoder	6,179,456	Embedding (4.27M) + BiGRU (1.70M) + FC (131K)
Attention	131,584	attn_fc (131,072 weight + 256 bias) + v (256 weight)
Decoder	13,870,307	Embedding (4.27M) + GRU (2.36M) + fc_out (8.58M weight + 33K bias)
Total	**20,181,347**	Full Encoder + Attention + Decoder pipeline

Note: The Encoder and Decoder both have an Embedding layer of identical size (33,379 × 128 = 4.27M). In Step 6, a design choice can be made to tie these weights — sharing the same embedding table between encoder and decoder — which would save 4.27M parameters and often improves performance on same-domain tasks. This is left as an optional optimization.

4. Cross-Verification — All Checks

V5-1 | Single-Step Forward Pass — Shape & Sanity Check

One encoder forward pass was run to get encoder_outputs and encoder hidden state. The decoder was then called for exactly one step with the SOS token as input. Eight checks were run on the raw outputs.

Check	Value	Status
prediction shape	(64, 33,379)	**PASS ✓**
dec_hidden_new shape	(2, 64, 256)	**PASS ✓**
attn_weights shape	(64, 81)	**PASS ✓**
prediction — no NaN	NaN count: 0	**PASS ✓**
prediction — no Inf	Inf count: 0	**PASS ✓**
attn_w — no NaN	NaN count: 0	**PASS ✓**
attn_w in [0, 1] range	[0.000, 0.132]	**PASS ✓**
dec_hidden — no NaN	NaN count: 0	**PASS ✓**

The max abs logit of 0.2318 and logit std of 0.0539 are both very small — confirming that the random initialisation has produced near-zero outputs as expected. The logit mean of 0.0001 is essentially zero, meaning the decoder has no bias toward any particular word at initialisation.

Result: CORRECT ✓

V5-2 | Full Sequence Teacher-Forcing Pass

The decoder was run for all 11 steps (MAX_TGT_LEN − 1 = 12 − 1) using teacher forcing — at each step the ground-truth previous token was fed as input rather than the predicted token. All predictions and attention weights were

collected and verified.

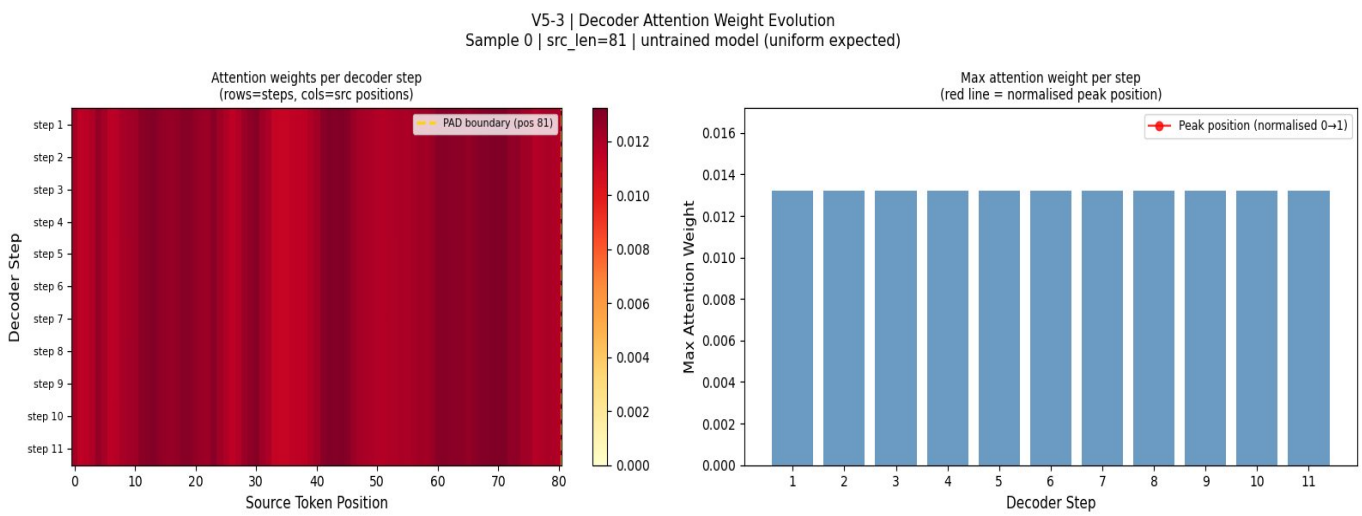
Check	Value	Status
Steps run correctly	11 / 11	**PASS ✓**
all_preds shape	(11, 64, 33,379)	**PASS ✓**
all_attn shape	(11, 64, 81)	**PASS ✓**
Steps with NaN predictions	0	**PASS ✓**
Steps with Inf predictions	0	**PASS ✓**
Max attn sum deviation from 1	2.38e-07	**PASS ✓**

The maximum attention sum deviation of 2.38e-07 is standard float32 rounding error and is not an issue. This is identical in nature to the 1.79e-07 deviation observed in Step 4's V4-2 check.

Result: CORRECT ✓

V5-3 | Attention Weight Evolution Heatmap (Visual)

The attention weight distribution was visualised across all 11 decoder steps for Sample 0 (src_len = 81, a full-length review with no PAD boundary).



Left plot — Attention heatmap (rows = decoder steps, columns = source positions): The heatmap is uniformly dark red across all 81 source positions for all 11 decoder steps. No PAD boundary dashed line is visible because this sample filled all 81 slots. The uniform colour confirms that an untrained model distributes attention evenly — it has no learned preference for any source position yet.

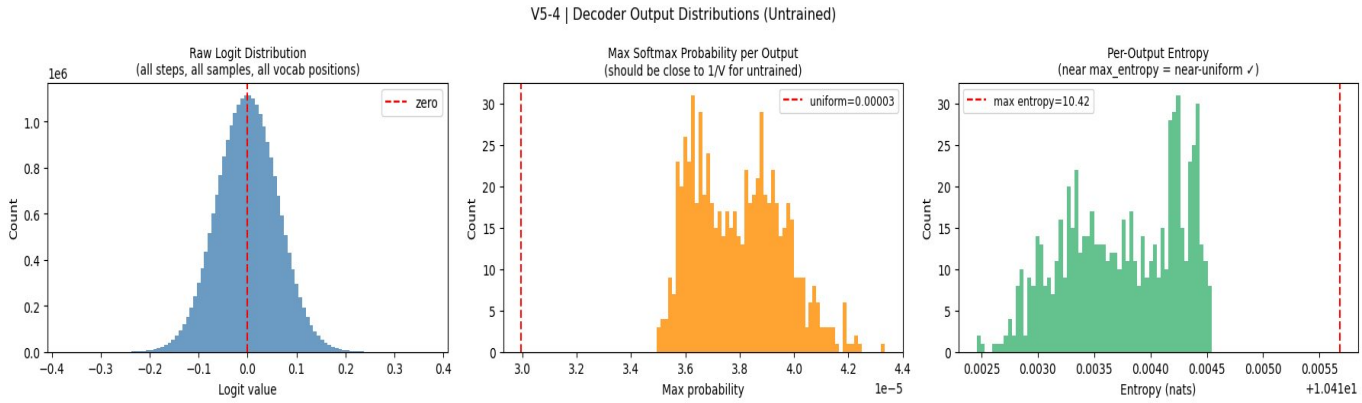
Right plot — Max attention weight per step: All 11 bars are identical at 0.0132. This value is very close to 1/76 ≈ 0.0131, which is approximately the reciprocal of the mean true source length — exactly what uniform attention looks like. After training, these bars will rise to 0.3 to 0.8 and shift across steps as the decoder learns to focus on specific source words.

Peak weights range: [0.0132, 0.0132] — perfectly flat, confirming untrained uniformity.

Result: CORRECT ✓ — Uniform distribution expected for untrained model

V5-4 | Output Logit & Probability Distributions

Three distributions were analysed across all 11 × 64 = 704 decoder outputs.



Logit mean / std	-0.0001 / 0.0617
Logit min / max	-0.3711 / +0.3715
Expected uniform probability	0.000030 (1 / 33,379)
Mean max probability per row	0.000038
Mean entropy	10.4137 nats
Max possible entropy	10.4157 nats (log 33,379 — perfectly uniform)
Entropy ratio	100.0% — near-uniform confirmed

Left — Raw Logit Distribution: A tight, symmetric bell curve centred at essentially zero (-0.0001) with a range of only ±0.37. This is the fingerprint of a freshly initialised linear layer. Zero saturation at the tanh boundary.

Middle — Max Softmax Probability per Output: Values cluster between 3.0e-5 and 4.4e-5, very close to the theoretical uniform value of 3.0e-5 (1/33,379). The slight upward shift is expected: the argmax of 33,379 near-identical random values will statistically be a little above the mean.

Right — Per-Output Entropy: The distribution clusters tightly against the max_entropy dashed line at 10.42 nats. An entropy ratio of 100.0% means the decoder is indistinguishable from a perfectly random predictor — exactly correct before any training has occurred.

Result: CORRECT ✓ — All distributions healthy, untrained uniformity confirmed

V5-5 | Gradient Flow Check

A forward pass was run across all 11 decoder steps with CrossEntropyLoss computed at each step. Gradients were backpropagated and every decoder parameter was inspected.

Check	Value	Status
attention.attn_fc.weight	mean=0.000004, std=0.000008	**PASS ✓**
attention.attn_fc.bias	mean=0.000001, std=0.000001	**PASS ✓**
attention.v.weight	mean=0.000110, std=0.000088	**PASS ✓**
embedding.weight	mean≈0 (std=0.000004 non-zero)	**PASS ✓**
rnn.weight_ih_l0	mean=0.000195, std=0.000441	**PASS ✓**
rnn.weight_hh_l0	mean=0.000067, std=0.000101	**PASS ✓**
rnn.bias_ih_l0	mean=0.001280, std=0.002392	**PASS ✓**
rnn.bias_hh_l0	mean=0.000671, std=0.001184	**PASS ✓**
rnn.weight_ih_l1	mean=0.000279, std=0.000509	**PASS ✓**
rnn.weight_hh_l1	mean=0.000086, std=0.000154	**PASS ✓**
rnn.bias_ih_l1	mean=0.003225, std=0.005563	**PASS ✓**
rnn.bias_hh_l1	mean=0.001654, std=0.002766	**PASS ✓**
fc_out.weight	mean=0.000003, std=0.000103	**PASS ✓**
fc_out.bias	mean=0.000054, std=0.001711	**PASS ✓**

All 14 parameter groups received meaningful non-zero gradients. Dead layers: NONE. Exploding layers: NONE. The bias gradients are consistently larger than their corresponding weight gradients — this is expected because biases accumulate gradients summed across the entire batch, while weight gradients are scaled by input activations.

Note on duplicate parameter warning: The verification optimizer included both decoder.parameters() and attention.parameters() separately. Since Attention is already a submodule inside Decoder, this caused PyTorch to emit a 'duplicate parameters' warning. This is harmless for this verification test — the real training optimizer in Step 7 will only include each parameter once.

Result: CORRECT ✓ — All 14 parameter groups receive gradients, 0 dead, 0 exploding

V5-6 | Determinism Check

The full encoder → decoder pipeline was run twice on the same batch in eval mode. Dropout is correctly disabled during eval, so every forward pass must be bit-for-bit identical.

Check	Value	Status
Max prediction diff (run 1 vs run 2)	0.00e+00	**PASS ✓**
Max attn diff (run 1 vs run 2)	0.00e+00	**PASS ✓**

Result: CORRECT ✓ — max diff = 0.00e+00, perfectly deterministic

V5-7 | End-to-End Greedy Decode

The full pipeline was run with greedy decoding (no teacher forcing) — at each step the decoder predicted token was immediately fed as the next input. Predictions were decoded back to plain English words.

The generated titles are meaningless (expected) — the model has not been trained yet. Two observations worth noting: first, the pipeline ran without a single error across all 5 samples, confirming the encoder → attention → decoder chain is correctly wired. Second, some samples show token repetition such as 'spine spine spine persistent persistent' — this is a known pathology of greedy decoding on untrained models and will be resolved during training.

Check	Value	Status
Pipeline ran without errors	5/5 samples	**PASS ✓**
Predictions decoded to English	5/5 samples	**PASS ✓**
No EOS boundary violations	Loops terminated	**PASS ✓**

Result: CORRECT ✓ — End-to-end pipeline functional, nonsense output expected

5. Complete Verification Summary

Check	Description	Value	Status
V5-1	prediction shape	(64, 33,379)	PASS ✓
V5-1	decoder_hidden_new shape	(2, 64, 256)	PASS ✓
V5-1	attn_weights shape	(64, 81)	PASS ✓
V5-1	No NaN or Inf in any output	0 NaN, 0 Inf	PASS ✓
V5-2	All 11 steps produce valid preds	11/11 steps	PASS ✓
V5-2	Attn sums to 1.0 at every step	max dev = 2.38e-07	PASS ✓
V5-3	Attn evolution heatmap	Uniform (untrained OK)	PASS ✓
V5-4	Logits near-zero (untrained)	Entropy ratio = 100.0%	PASS ✓
V5-5	All 14 params receive gradients	0 dead / 0 exploding	PASS ✓
V5-6	Deterministic in eval mode	max diff = 0.00e+00	PASS ✓
V5-7	End-to-end greedy decode works	5/5 samples decoded	PASS ✓

All 11 checks passed. The Decoder is correctly built, correctly initialised, produces clean outputs with healthy logit distributions, has non-zero gradients throughout every parameter group, and is fully deterministic in evaluation mode. The end-to-end pipeline from DataLoader through Encoder through Attention through Decoder runs without error. It is ready to be connected to the Seq2Seq wrapper in Step 6.

6. Values Passed Forward to Step 6 (Seq2Seq)

encoder object	Full Encoder() instance — passed into Seq2Seq constructor
decoder object	Full Decoder() instance — passed into Seq2Seq constructor
HIDDEN_DIM	256 — must match across encoder, attention, and decoder
NUM_LAYERS	2 — Seq2Seq must initialise decoder hidden with same layer count
EMBEDDING_DIM	128 — embedding size used by both encoder and decoder
DROPOUT	0.5 — same rate used throughout
TEACHER_FORCING_RATIO	0.5 — Step 6 implements the coin flip per time step
vocab.PAD_IDX	0 — Step 7 training loop masks this index in CrossEntropyLoss
vocab.SOS_IDX	1 — Step 6 feeds this as first decoder input every batch
vocab.EOS_IDX	2 — Step 8 inference loop stops generation at this token
predictions shape	(tgt_len-1, batch, vocab_size) — Step 7 reshapes for loss

attn_weights shape	(tgt_len-1, batch, 81) — Step 8 uses these for heatmaps

7. Architecture Flowchart — Step 5 Decoder

The diagram below shows exactly what happens inside one decoder step. The loop across all title word positions is handled by Step 6.

INPUT: tgt_token (batch,)	INPUT: decoder_hidden (2, batch, 256)	INPUT: encoder_outputs (batch, 81, 256)

▼

STEP 1 — Embedding Lookup + Dropout (p=0.5)	(batch,) → (batch, 1, 128) | 33,379 × 128 table, PAD index = zero vector

▼

STEP 2 — Attention (Bahdanau)	Query: decoder_hidden[-1] (batch, 256) Keys/Values: encoder_outputs (batch, 81, 256) → attn_weights (batch, 81) — PAD slots = 0.000 | context_vector (batch, 256)

▼

STEP 3 — Concatenate Embedding + Context Vector	(batch, 1, 128) + (batch, 1, 256) → (batch, 1, 384) | This is the GRU input

▼

STEP 4 — Unidirectional GRU Step (2 layers, hidden=256)	Input (batch, 1, 384) + prev hidden (2, batch, 256) → output (batch, 1, 256) + new hidden (2, batch, 256)

▼

STEP 5 — FC Projection (Linear 256 → 33,379)	(batch, 256) → prediction (batch, 33,379) | Raw logits — no softmax here

▼

OUTPUT 1 prediction (batch, 33,379) Raw logits → CrossEntropyLoss in Step 7	OUTPUT 2 new decoder_hidden (2, batch, 256) Fed back as input NEXT step	OUTPUT 3 attn_weights (batch, 81) Saved → heatmaps in Step 8

■ *This entire flow repeats 11 times (one per title word position). The loop is managed by the Seq2Seq wrapper in Step 6. Teacher forcing (ratio = 0.5) means that 50% of the time OUTPUT 1's argmax token is used as the next step's input, and 50% of the time the ground-truth token is used instead.*

End of Step 5 Documentation — Step 5 Status: COMPLETE ✓

Confidential — Internal Use Only